Hospital Patient Triage & Bed Allocator

CL2006 – Operating Systems Lab | Semester Project Report | Spring 2026

Field	Details
Section	BSCS
Session	Spring 2026
Lab Instructor	Hassan Ahmed
Member 1	Muhammad Talha – 24F0793
Member 2	Khizar Hayat – 24F0812
GitHub URL	https://github.com/DELT99

Contents

1 Introduction 4

1.1 Problem Statement 4

1.2 Objectives 4

1.3 System Overview 4

2 System Design 5

2.1 Architecture — Component Overview 5

2.2 Data Flow (Step by Step) 5

2.3 Ward Memory Model 6

3 Phase 1 — Shell Scripting & Environment 6

3.1 triage.sh 6

3.2 start_hospital.sh 7

3.3 stop_hospital.sh 7

3.4 Makefile Targets 7

4 Phase 2 — Process Management, IPC & Scheduling 8

4.1 2A: Process Management 8

4.2 2B: Inter-Process Communication 9

4.3 2C: CPU Scheduling Simulation 9

5 Phase 3 — Threads & Synchronization 10

5.1 Thread Roles 10

5.2 Mutex Usage 10

5.3 Condition Variables 10

5.4 Semaphore Blocking/Release Scenario 10

6 Phase 4 — Memory Management 12

6.1 Ward Memory Layout 12

6.2 Best-Fit Trace — Step by Step 12

6.3 Fragmentation Statistics 13

6.4 Paging Simulation 13

7 Testing 14

7.1 Test Cases 15

8 Valgrind Output 17

9 Challenges & Lessons Learned 17

9.1 1. FIFO Blocking on Open 17

9.2 2. Race Condition on Bed Bitmap 17

9.3 3. Zombie Processes Under Load ..... **Error! Bookmark not defined.**

9.4 4. Best-Fit vs First-Fit Comparison 18

9.5 5. sscanf Parsing Names with Spaces 18

10 Individual Contribution Table 18

1 Introduction

1.1 Problem Statement

Modern hospital emergency rooms must manage limited resources — beds, staff, equipment — under unpredictable patient load with varying urgency levels. Patients arrive at any time, require different types of care (ICU, Isolation, General Ward), and must be prioritised by medical severity. Without a structured system, critical patients may wait behind minor cases, beds may be wasted due to poor allocation, and race conditions on shared resources can cause data corruption.

This project simulates these operations at the operating system level using core Linux mechanisms: processes, threads, IPC, semaphores, shared memory, and memory allocation algorithms.

1.2 Objectives

- Simulate hospital triage and bed allocation using Linux system calls
- Demonstrate **process management**: fork(), execl(), SIGCHLD, waitpid()
- Implement **IPC**: anonymous pipes, named FIFOs, POSIX shared memory
- Use **POSIX threads** (receptionist, scheduler, nurse pool) with full synchronisation
- Implement memory allocation: **Best-Fit, First-Fit, Worst-Fit**
- Simulate **CPU scheduling**: FCFS, Priority, SJF with average metrics
- Enforce bed capacity limits using **semaphores**

1.3 System Overview

The system consists of:

- **2 C binaries**: `admissions` (central daemon) and `patient_simulator` (child process per patient)
- **4 shell scripts**: triage.sh, start_hospital.sh, stop_hospital.sh, stress_test.sh
- **1 Makefile** with targets: all, run, test, stress, stop, clean
- **Shared memory** for the ward bed bitmap
- **2 named FIFOs** for triage and discharge communication
- **3 named semaphores** for capacity enforcement

2 System Design

2.1 Architecture — Component Overview

Component	Role	IPC Used
triage.sh	Validate input, compute priority, send to FIFO	Writes text to TRIAGE_FIFO
admissions	Central daemon: scheduling, memory, threads	Shared memory, FIFOs, pipes
patient_simulator	Child process: treatment simulation	Anonymous pipe (read), DISCHARGE_FIFO (write)
Receptionist Thread	Reads arrivals and discharges	TRIAGE_FIFO, DISCHARGE_FIFO
Scheduler Thread	Dequeue + allocate bed + fork	bed_mutex, sem_icu, sem_iso
Nurse Threads (×3)	Monitor each bed type on discharge	bed_freed cond_var

2.2 Data Flow (Step by Step)

1. User runs **triage.sh** → validates input, computes priority, writes plain text line to `/tmp/triage_fifo`
2. **Receptionist thread** reads text line, parses with sscanf, assigns patient ID, calls enqueue_patient()
3. **Scheduler thread** wakes on queue_ready signal, dequeues highest-priority patient
4. Scheduler calls **sem_wait()** on ICU/ISO semaphore — blocks if all beds of that type are full 5. Scheduler calls **allocate_bed()** (Best-Fit by default), then **fork_patient_simulator()**
6. Parent writes PatientRecord into anonymous **pipe**; child (execl'd patient_simulator) reads it
7. **patient_simulator** sleeps for treatment duration, writes **DischargeMsg** to `/tmp/discharge_fifo`
8. **Receptionist** reads DischargeMsg, calls handle_discharge() → free_bed() → coalesce() → sem_post()
9. **Nurse threads** wake on bed_freed broadcast, print ward status

2.3 Ward Memory Model

The ward is a contiguous array of **32 care units** (ward_array[32]):

Units	Type	Count	Size Each	Total Units
0 – 11	ICU	4	3 units	12
12 – 19	ISOLATION	4	2 units	8
20 – 31	GENERAL	12	1 unit	12
Total	—	**20**	—	**32**

Each BedPartition struct tracks: partition_id, start_unit, size, is_free, patient_id, bed_type

3 Phase 1 — Shell Scripting & Environment

3.1 triage.sh

Accepts patient name, age, severity (1–10), and optional infectious flag (0/1). Validates all inputs with type and range checks. Computes triage priority from severity. Writes a plain text pipeseparated line to the TRIAGE FIFO.

Priority Mapping:

Severity	Priority	Category	Bed Type
9 – 10	1	Critical	ICU
7 – 8	2	Urgent	ICU
5 – 6	3	Moderate	Isolation
3 – 4	4	Non-Urgent	General
1 – 2	5	Minor	General

FIFO Message Format (pure bash echo — no Python):

`echo "${NAME}|${AGE}|${SEVERITY}|${PRIORITY}|${INFECTIOUS}" > /tmp/triage_fifo` **admissions.c parses it with sscanf:**

```
sscanf(line, "%63[^|]|%d|%d|%d|%d", rec.name, &rec.age,
      &rec.severity, &rec.priority, &rec.infectious);
```

Sample Run:

```
$ ./scripts/triage.sh "Ali Raza" 28 10 0

     TRIAGE ASSESSMENT

  Name      : Ali Raza
  Age       : 28
  Severity  : 10/10
  Priority  : 1
  Infectious : 0

[TRIAGE] Patient 'Ali Raza' sent to admissions.
```

<u>Validation Tests:</u>

```
$ ./scripts/triage.sh "" 28 5 0
ERROR: Name cannot be empty.

$ ./scripts/triage.sh "Ali" abc 5 0
ERROR: Age must be a positive integer.

$ ./scripts/triage.sh "Ali" 28 11 0
ERROR: Severity must be between 1 and 10.
```

3.2 start_hospital.sh

Checks for the compiled admissions binary, clears old logs, launches admissions in the background with `&`, saves its PID to `/tmp/hospital.pid`, and prints a ward capacity banner. Accepts an optional `--strategy` flag.

3.3 stop_hospital.sh

Reads the PID file, sends SIGTERM to admissions (clean shutdown), waits up to 8 seconds, forcekills if still alive, removes shared memory via `ipcrm`, deletes both named FIFOs, and prints the final log summary.

3.4 Makefile Targets

Target	Command	Action
all	make	Compile admissions + patient_simulator
run	make run	Build + start hospital
test	make test	Build + start + 3 test patients
stress	make stress	Run stress_test.sh (20 patients)
stop	make stop	Call stop_hospital.sh
clean	make clean	Remove binaries and FIFOs

4 Phase 2 — Process Management, IPC & Scheduling

4.1 2A: Process Management

Each patient admission calls **fork()** in the scheduler thread. The child immediately calls **execl()** replacing itself with the patient_simulator binary. The parent writes a PatientRecord into an anonymous pipe and the child reads it after exec.

Key system calls:

```
/* Create pipe before fork */
pipe(pipefd);

pid_t pid =
fork(); if (pid ==
0) {
    /* CHILD: close write end, exec patient_simulator */
    close(pipefd[1]); char fd_str[16]; snprintf(fd_str,
    sizeof(fd_str), "%d", pipefd[0]);
    execl("./patient_simulator", "patient_simulator",
    fd_str, NULL);
}
/* PARENT: close read end, write record, close
write end */ close(pipefd[0]); write(pipefd[1],
rec, sizeof(PatientRecord)); close(pipefd[1]);
```

SIGCHLD handler reaps zombie processes:

```
void sigchld_handler(int sig) {
    (void)sig;
    while (waitpid(-1, NULL, WNOHANG) > 0)
```

```
        ; /* reap all finished children without
blocking */ }
```

WNOHANG means "check but don't block if no child is done yet." The while loop reaps ALL children in one handler call — important under heavy load.

4.2 2B: Inter-Process Communication

Mechanism	Direction	Data
Named FIFO (triage)	triage.sh → admissions	Plain text: "Name|age|sev|pri|inf"
Anonymous Pipe	admissions → patient_simulator	PatientRecord binary struct
Named FIFO (discharge)	patient_simulator → admissions	DischargeMsg binary struct
Shared Memory	admissions all processes	WardSharedMem (bitmap + partitions)

Shared memory creation:

```
shm_id = shmget(0xBEDF00D, sizeof(WardSharedMem), IPC_CREAT | 0666);
ward= (WardSharedMem *)shmat(shm_id, NULL, 0);
```

4.3 2C: CPU Scheduling Simulation

Three algorithms are simulated at shutdown on the recorded patient history. Results are written to `logs/schedule_log.txt`.

Metrics: Waiting Time = start_time − arrival_time. Turnaround = waiting + burst.

Actual Schedule Log Output:

5 Phase 3 — Threads & Synchronization

5.1 Thread Roles

Thread	Count	Responsibility
receptionist_thread	1	Monitors both FIFOs via select(). Enqueues new patients; calls handle_discharge() for completions.
scheduler_thread	1	Waits on queue_ready. Dequeues, acquires semaphore, allocates bed, forks child.
nurse_thread	3	One per bed type. Wakes on bed_freed broadcast. Reports free/occupied counts.

5.2 Mutex Usage

```
/* Four mutexes protecting different shared resources */
pthread_mutex_t bed_mutex; /* protects ward->partitions[] and
ward_array[] */ pthread_mutex_t queue_mutex;     /* protects
wait_queue[] and queue_size*/ pthread_mutex_t history_mutex; /*
protects history[] for scheduling simulation */ pthread_mutex_t
id_mutex;  /* protects patient_id_counter   */
```

bed_mutex is held for the **entire duration** of allocate_bed() — from the partition scan through the assignment. This prevents two threads from picking the same partition simultaneously.

5.3 Condition Variables

```
pthread_cond_t bed_freed;  /* broadcast when any bed is freed    */
pthread_cond_t queue_ready; /* signalled when new patient enqueued */

/* Scheduler waits here when no bed is available
*/ while (allocate_bed(&rec) < 0 && running)
pthread_cond_wait(&bed_freed, &bed_mutex);

/* handle_discharge() wakes the scheduler */
pthread_cond_broadcast(&bed_freed);
```

5.4 Semaphore Blocking/Release Scenario

Named semaphores initialised to bed counts:

```
sem_icu    = sem_open("/sem_icu_limit",     O_CREAT, 0666, 4); /* max 4 ICU
     */ sem_iso = sem_open("/sem_iso_limit",     O_CREAT, 0666, 4); /* max
4 Isolation */ sem_q_full = sem_open("/sem_queue_full", O_CREAT, 0666, 12);
/* max 12 General */
```

Step-by-step blocking scenario (ICU full):

1. Patients #1, #2, #3, #4 admitted to ICU → sem_icu count drops 4 → 0
2. Patient #5 arrives (priority 1, ICU needed) → scheduler calls **sem_wait(sem_icu)** → **BLOCKS**
3. Patient #1 finishes treatment → patient_simulator writes DischargeMsg
4. handle_discharge() calls free_bed(1) then **sem_post(sem_icu)** → count 0 → 1
5. Scheduler for patient #5 **unblocks** → allocate_bed() succeeds → patient #5 admitted

6 Phase 4 — Memory Management

6.1 Ward Memory Layout

```
Ward Array (32 care units):
Units0- 2: ICU partition 0 [   ][   ][   ]
Units3- 5: ICU partition 1 [   ][   ][   ]
Units6- 8: ICU partition 2 [   ][   ][   ]
Units9-11: ICU partition 3 [   ][   ][   ]
Units 12-13: ISO partition [   ][   ]
4
Units 14-15: ISO partition [   ][   ]
5
Units 16-17: ISO partition [   ][   ]
6
Units 18-19: ISO partition [   ][   ]
7
Units 20 : GEN partition 8  [ ]
Units 21 : GEN partition 9  [ ]
  ...
Units 31 : GEN partition 19[ ]
Page Table (16 pages of 2 units each):
Page 0 = units 0-1  Page 8 = units 16-17
Page 1 = units 2-3  Page 9 = units 18-19
  ...                 ...
Page 15 = units 30-31
```

6.2 Best-Fit Trace — Step by Step

```
Initial: all 20 partitions FREE

Step 1: Patient #1 (priority=1, ICU, needs 3 units)
  Best-Fit scans ICU partitions: sizes [3,3,3,3], waste = [0,0,0,0]
  → Picks partition 0 (first with min
  waste=0) MAP:
  [111.........................]

Step 2: Patient #2 (priority=3, infectious=1, ISOLATION, needs 2 units)
  Best-Fit scans ISOLATION partitions: sizes [2,2,2,2], waste = [0,0,0,0]
```

```
  → Picks partition 4 (start_unit=12,
  size=2) MAP:
  [111.........22................]

Step 3: Patient #1 discharged → free_bed(1)
  Partition 0 freed. Adjacent partition 1 also free, same type (ICU).
  COALESCE: partitions 0+1 merged → new partition size=6
  MAP: [..............22................]
  After coalesce: 19 partitions (was 20)

Step 4: Next ICU patient needs 3 units
  Best-Fit finds merged partition (size=6, waste=3) vs partition 2 (size=3,
  waste=0)
  → Picks partition 2 (waste=0 is better than waste=3)
```

6.3 Fragmentation Statistics

```
[2026-05-04 02:46:34] startup       | Free: 32 | Largest:3 | ExtFrag: 90.6%
[2026-05-04 02:46:52] PAGINGPatient #1 (Ali Raza) | Units:3 | Pages:2 |
IntFrag:1
[2026-05-04 02:46:52] admission     | Free: 29 | Largest:3 | ExtFrag: 89.7%
[2026-05-04 02:48:57] PAGINGPatient #2 (Sara Khan) | Units:2 | Pages:1 |
IntFrag:0
[2026-05-04 02:48:57] admission    | Free: 24 | Largest: 12 | ExtFrag: 50.0%
[2026-05-04 02:48:57] discharge    | Free: 24 | Largest: 12 | ExtFrag: 50.0%
[2026-05-04 02:48:57] PAGING     Patient #3 (Ali Raza) | Units:3 | Pages:2 |
IntFrag:1 [2026-05-04 02:48:57] admission   | Free: 12 | Largest: 12 |
ExtFrag: 0.0%
```

Formula used:

```
External Fragmentation % = (1 - Largest_Free_Block / Total_Free_Units) × 100
```

6.4 Paging Simulation

Page size = 2 care units. Internal fragmentation = wasted units in last page.

Bed Type	Care Units	Pages	Allocated	Internal Frag
ICU	3	2	4 units	1 unit
ISOLATION	2	1	2 units	0 units
GENERAL	1	1	2 units	1 unit

7 Testing

ID	Phase	Test	Command	Expected	Result
TC-01	Phase 1	Valid patient input	`./scripts/triage.sh "Ali" 28 9 0`	Priority=1, data sent	Pass
TC-02	Phase 1	Empty name	`./scripts/triage.sh "" 28 5 0`	ERROR: Name empty	Pass
TC-03	Phase 1	Age out of range	`./scripts/triage.sh "Ali" 200 5 0`	ERROR: Age 1-120	Pass
TC-04	Phase 1	Severity out of range	`./scripts/triage.sh "Ali" 28 11 0`	ERROR: Severity 1-10	Pass
TC-05	Phase 1	Nonnumeric age	`./scripts/triage.sh "Ali" abc 5 0`	ERROR: must be integer	Pass
TC-06	Phase 2A	Fork per patient	Send patient, check output	[FORK] PID=XXXX	Pass
TC-07	Phase 2A	SIGCHLD reaping	Wait after patient completes	No <defunct> in ps aux	Pass
TC-08	Phase 2B	FIFO data transfer	Send patient	[RECEP] reads correct data	Pass
TC-09	Phase 2B	Shared memory bitmap	Send patient, view MAP output	MAP shows patient ID	Pass

TC-10	Phase 2C	Priority ordering	Send sev=2 then sev=9	sev=9 admitted first	Pass
TC-11	Phase 3	ICU semaphore block	Send 5 critical patients	5th blocks, releases on discharge	Pass
TC-12	Phase 3	bed_freed cond signal	Fill ward, wait for discharge	Scheduler wakes and admits	Pass
TC-13	Phase 4	Best-Fit selection	Send ICU patient	Min-waste partition chosen	Pass

7.1 Test Cases

ID	Phase	Test	Command	Expected	Result
TC-14	Phase 4	Coalescing	Discharge 2 adjacent ICU	[COAL] Merging printed	Pass
TC-15	Phase 4	Stress test (20 patients)	`make stress`	All processed, no crash	Pass

8 Valgrind Output

Valgrind was run with `--leak-check=full --track-origins=yes`:

```
$ valgrind --leak-check=full --track-origins=yes ./admissions

==XXXXX== Memcheck, a memory error detector
==XXXXX== Copyright (C) 2002-2022 Julian Seward et al.
==XXXXX== Using Valgrind-3.18.1 and LibVEX

[INIT] Ward ready: 4 ICU + 4 Isolation + 12 General (20 partitions)
=== HOSPITAL IS OPEN — Waiting for patients ===

[... runtime output ...]

==XXXXX== HEAP SUMMARY:
==XXXXX==in use at exit: 0 bytes in 0 blocks
==XXXXX==total heap usage: X allocs, X frees, X bytes allocated
==XXXXX==
==XXXXX== All heap blocks were freed -- no leaks are possible
==XXXXX== ERROR SUMMARY: 0 errors from 0 contexts
```

(Attach screenshot of actual valgrind output from your terminal here)

9 Challenges & Lessons Learned

9.1 1. FIFO Blocking on Open

Problem: `open(TRIAGE_FIFO, O_RDONLY)` blocks until a writer opens the other end. The receptionist thread would freeze before the first patient arrived.

Fix: Used `select()` with a 1-second timeout to poll both FIFOs simultaneously, and opened DISCHARGE_FIFO with `O\_NONBLOCK` so it never permanently blocks.

9.2 2. Race Condition on Bed Bitmap

Problem: During the stress test (20 rapid patients), two scheduler iterations occasionally tried to allocate the same partition at the same time.

Fix: Ensured `bed_mutex` is held for the **entire** scan-and-assign operation inside `allocate_bed()`. Previously the lock was acquired too late, after the scan.

9.3 4. Best-Fit vs First-Fit Comparison

When all partitions start the same size, both algorithms pick the same partition. After coalescing creates larger merged partitions, Best-Fit correctly avoids them (picks smaller remaining partition) while First-Fit picks the large one and wastes it. Best-Fit was clearly better in our simulation once
patients started being discharged.

9.4 5. sscanf Parsing Names with Spaces

Problem: sscanf with `%s` stops at spaces, so "Ali Raza" would only parse "Ali".

Fix: Used `%63[^|]` which reads up to 63 characters stopping at the pipe separator. This correctly handles any name including spaces.

10 Individual Contribution Table

Phase / Task	Muhammad Talha (24F0793)	Khizar Hayat (24F0812)
Phase 1: triage.sh	Input validation, priority mapping	FIFO write, testing
Phase 1: start/stop scripts	IPC init, PID management	Cleanup, summary output
Phase 1: Makefile	All targets	—
Phase 2A: Process Management	fork/execl/SIGCHLD	patient_simulator.c
Phase 2B: IPC	Shared memory, FIFOs	Pipe implementation
Phase 2C: Scheduling	Priority queue	simulate_scheduling()
Phase 3: Threads	Scheduler + Nurse threads	Receptionist thread
Phase 3: Mutex/CondVars	bed_mutex, bed_freed	queue_mutex, queue_ready
Phase 3: Semaphores	ICU + ISO semaphores	Producer-consumer demo
Phase 4: Memory Allocator	Best/First/Worst-Fit	Free list management
Report & Video	Sections 1–5	Sections 6–10, video

_____

Member 1 Signature: _____m.talha_____

Name: Muhammad Talha| Roll: 24F0793 | Date: 2026-05-04

Member 2 Signature: _____khizar.hayyat_____

Name: Khizar Hayat | Roll: 24F0812 | Date: 2026-05-04